

Agenda

- Java IO framework
- GC
- enum
- Reflection

PrintStream class

- Produce formatted output (in bytes) and send to underlying stream.
- Formatted output is done using methods print(), println(), and printf().
- System.out and System.err are objects of PrintStream class.
- It is used only to write the formatted data in to the file.

```
List<Movie> movies = new ArrayList<>();
movies.add(new Movie(1, "Star wars", 7.5));
movies.add(new Movie(2, "God Father", 8.0));
movies.add(new Movie(3, "Harry Potter", 7.0));
movies.add(new Movie(4, "Bruce Almighty", 6.5));
movies.add(new Movie(5, "Forest Gump", 8.5));

try (FileOutputStream fout = new FileOutputStream("movies4.txt")) {
    try (PrintStream pout = new PrintStream(fout)) {
        for (Movie m : movies)
            pout.printf("%5d|%-20s|%.1f\n", m.getId(), m.getTitle(), m.getRating());
    }
}
```

Scanner class

- Added in Java 5 to get the formatted input.
- It is java.util package (not part of java io framework).

```
Scanner sc = new Scanner(inputStream);
// OR
Scanner sc = new Scanner(inputFile);
```

- Helpful to read text files line by line.

```
public static final String FILE = "movies.txt";

public static Movie parseMovie(String line) {
    String[] parts = line.split(",");
    int id = Integer.parseInt(parts[0]);
    String title = parts[1];
    double rating = Double.parseDouble(parts[2]);
}
```

```
Movie movie = new Movie(id,title,rating);
return movie;
}

public static void main(String[] args) throws IOException {
List<Movie> movies = new ArrayList<>();
try (FileInputStream fin = new FileInputStream(FILE)) {
    try (Scanner sc = new Scanner(fin)) {
        while (sc.hasNextLine()) {
            String line = sc.nextLine();
            Movie movie = parseMovie(line);
            movies.add(movie);
        }
    }
}
}
```

Character encoding

- Words and sentences in text are created from characters.
- Examples of characters include the Latin letter á or the Chinese ideograph 請 or the Devanagari character ह्र.
- Characters that are needed for a specific purpose are grouped into a character set.
- Each character is associated with a number, called a code point
- The characters are stored in the computer as one or more bytes.
- A character encoding provides a key to unlock (ie. crack) the code.
- It is a set of mappings between the bytes in the computer and the characters in the character set. Without the key, the data looks like garbage.
- Unfortunately, there are many different character sets and character encodings, ie. many different ways of mapping between bytes, code points and characters.

Types of Character Sets / Encodings

1.ASCII (American Standard Code for Information Interchange)

- Most basic and oldest encoding
- Features
 - Uses 7 bits
 - Supports 128 characters
 - Mainly supports English characters

Character	ASCII Value
A	65
B	66
a	97
0	48

Character	ASCII Value
Space	32

- Limitation
 - Cannot store characters like: Hindi, Chinese

2. Unicode (Most Important Modern Encoding)

- Unicode solves language limitation problems.
- Features
 - Supports characters from almost all languages
 - Supports symbols and emojis
 - Each character has a unique number

Character	Unicode
A	U+0041
₹	U+20B9
😊	U+1F600

Unicode Encoding Formats

1. UTF-8 (Most Popular)

- Used everywhere (Internet standard)
- Features
 - Variable length encoding
 - Uses 1 to 4 bytes
 - Backward compatible with ASCII
 - Saves memory
 - Supports all languages
 - Web standard

Character	Bytes Used
A	1 byte
₹	3 bytes
😊	4 bytes

2. UTF-16

- Used in Windows and Java internally
- Features
 - Uses 2 or 4 bytes

3.UTF-32

- Features
 - Uses fixed 4 bytes for every character
 - Simple but uses more memory

Character streams

- Character streams are used to interact with text file.
- Java char takes 2 bytes (unicode), however char stored in disk file may take 1 or more bytes depending on char encoding.
 - <https://www.w3.org/International/questions/qa-what-is-encoding>
- The character stream does conversion from java char to byte representation and vice-versa (as per char encoding).
- The abstract base classes for the character streams are the Reader and Writer class.
- Writer class -- write operation
 - void close() -- close the stream
 - void flush() -- writes data (in memory) to underlying stream/device.
 - void write(char[] b) -- writes char array to underlying stream/device.
 - void write(int b) -- writes a char to underlying stream/device.
- Writer Sub-classes
 - FileWriter, OutputStreamWriter, PrintWriter, BufferedWriter, etc.
- Reader class -- read operation
 - void close() -- close the stream
 - int read(char[] b) -- reads char array from underlying stream/device
 - int read() -- reads a char from the underlying device/stream. Returns -1
- Reader Sub-classes
 - FileReader, InputStreamReader, BufferedReader, etc.

```
try(FileReader rd = new FileReader("file1.txt"))
{
    int ch;
    while((ch=rd.read())!=-1) {
        System.out.print((char)ch);
    }
}
```

- Compile using jdk-11 compiler as the FileReader ctor with Charset is defined in java 11 version

```
try (FileReader rd = new FileReader("file2.txt", Charset.forName("UTF-16LE"))) {
    int ch;
    while ((ch = rd.read()) != -1) {
        System.out.print((char) ch);
    }
}
```

Reflection

- It is a technique to read the metadata and work with that data.
- .class = Byte-code + Meta-data + Constant pool + ...
- When class is loaded into JVM all the metadata is stored in the object of java.lang.Class (heap area).
- This metadata includes class name, super class, super interfaces, fields (field name, field type, access modifier, flags), methods (method name, method return type, access modifier, flags, method arguments, ...), constructors (access modifier, flags, ctor arguments, ...), annotations (on class, fields, methods, ...).

Reflection applications

- Inspect the metadata (like javap)
- Build IDE/tools (Intellisense)
- Dynamically creating objects and invoking methods
- Access the private members of the class

Get the java.lang.Class object

- way 1: When you have class-name as a String (taken from user or in properties file)

```
Class<?> c = Class.forName(className);
```

- way 2: When the class is in project/classpath.

```
Class<?> c = ClassName.class;
```

- way 3: When you have object of the class.

```
Class<?> c = obj.getClass();
```